



**UNIVERSITÀ
DI TORINO**

Università degli Studi di Torino

Corso di Laurea in Informatica

Confronto tra LLM per la Generazione di Programmi eBPF

Tesi di Laurea

Relatore

Prof. Birke Robert René Maria

Correlatore

Dott. Mittone Gianluca

Candidato

Grillo Giovanni

M.989819

Anno Accademico 2024/2025

Abstract

Lo sviluppo di programmi eBPF richiede competenze approfondite di programmazione del kernel e progettazione di sistemi, fattori che ne limitano l'accessibilità. Questa tesi esplora l'utilizzo di modelli linguistici di grandi dimensioni (LLM) per automatizzare la generazione di programmi eBPF tramite bpfftrace, riducendo in modo significativo la barriera d'ingresso a questa tecnologia fondamentale per l'osservabilità e la sicurezza dei sistemi.

Il lavoro si basa su **Kgent** (Kernel Extensions Large Language Model Agent), un sistema agentico che combina retrieval semantico di esempi validati, few-shot learning e meccanismi di retry automatico guidati dal feedback di compilazione. Sono stati valutati cinque modelli LLM open-source di dimensioni diverse su un dataset completo che copre tracing kernel, system calls, networking, filesystem e analisi avanzate.

I risultati mostrano che i modelli più grandi sono in genere più affidabili, ma non è solo la loro dimensione a fare la differenza. L'architettura e la qualità dell'addestramento contano di più: alcuni modelli di media grandezza, se ben progettati e addestrati, possono ottenere risultati paragonabili a quelli di modelli molto più grandi.

L'analisi rivela dinamiche interessanti nel comportamento degli LLM: i modelli più compatti commettono prevalentemente errori sintattici ma dimostrano notevoli capacità di auto-correzione quando guidati dal feedback, mentre i modelli più grandi generano codice corretto al primo tentativo con errori occasionali di tipo logico. La performance varia inoltre significativamente tra categorie di task, evidenziando una forte dipendenza dalla rappresentazione nei dataset di addestramento.

Questo lavoro dimostra che l'automazione della creazione di codice eBPF tramite LLM open-source è concretamente realizzabile. Il risultato favorisce un uso più accessibile di questa tecnologia e fornisce indicazioni pratiche per scegliere il modello più adatto in base alle risorse disponibili, ai requisiti di affidabilità e al tipo di applicazione.

Dichiaro di essere responsabile del contenuto dell'elaborato che presento al fine del conseguimento del titolo, di non avere plagiato in tutto o in parte il lavoro prodotto da altri e di aver citato le fonti originali in modo congruente alle normative vigenti in materia di plagio e di diritto d'autore. Sono inoltre consapevole che nel caso la mia dichiarazione risultasse mendace, potrei incorrere nelle sanzioni previste dalla legge e la mia ammissione alla prova finale potrebbe essere negata.

Indice

Introduzione	5
1 Background	7
1.1 Large Language Models	7
1.1.1 Dalle Origini agli LLM	7
1.1.2 Architettura Transformer	8
1.1.3 Evoluzione dei Large Language Models	9
1.1.4 LLM per la Generazione di Codice	10
1.2 eBPF: Extended Berkeley Packet Filter	10
1.2.1 Storia ed Evoluzione	10
1.2.2 Architettura e Componenti	11
1.2.3 Casi d'Uso Principali	12
1.2.4 Limiti e Complessità	12
1.3 Agenti Intelligenti	12
2 Metodologia	15
2.1 Struttura	15
2.1.1 Panoramica e Obiettivi del Progetto	15
2.1.2 Architettura del Sistema	15
2.2 Problematiche nella Replicazione	18
2.2.1 Scoperta dei Vincoli Operativi	18
2.2.2 Decisione di Migrazione a Modelli Locali	18
2.3 Strategia di Migrazione ai Modelli Locali	19
2.3.1 Percorso di Adozione	19
2.3.2 Adozione di Ollama	19
2.3.3 Infrastruttura Multi-Modello	20
2.3.4 Migrazione degli Embeddings	20
2.4 Adattamenti Implementativi	20
2.4.1 Architettura Distribuita	20
2.4.2 Vantaggi del Modulo Centralizzato	23
2.4.3 Modifiche ai Componenti Core	24
2.4.4 Downgrade di bpftrace per Compatibilità	25
3 Risultati	27
3.1 Setup	27
3.1.1 Dataset di Test	27
3.1.2 Modelli Valutati	27
3.1.3 Metodologia di Valutazione	28
3.1.4 Parametri di Generazione	29

3.2	Risultati	30
3.2.1	Tipologia di errori	30
3.2.2	Effetto del retry	31
3.2.3	Performance per categoria di test	32
4	Conclusioni	33
	Conclusioni	33

Introduzione

Il monitoraggio e l'analisi delle prestazioni dei sistemi operativi rappresenta una sfida crescente nell'era dei sistemi distribuiti e del cloud computing. La necessità di ottenere visibilità approfondita sul comportamento del Kernel Linux senza compromettere le performance ha portato allo sviluppo di eBPF (extended Berkeley Packet Filter), una tecnologia che permette l'esecuzione sicura di programmi personalizzati all'interno del kernel.

eBPF consente di osservare eventi di sistema, tracciare funzioni kernel, analizzare il traffico di rete e implementare politiche di sicurezza con overhead minimo e senza modificare il codice sorgente del sistema operativo. Nonostante le notevoli potenzialità offerte da questa tecnologia, la sua diffusione risulta limitata dall'elevata complessità del suo utilizzo: lo sviluppo di programmi eBPF richiede una conoscenza approfondita dell'architettura del kernel Linux, della sintassi peculiare di eBPF e delle restrizioni imposte dal *verifier*, necessarie a garantire la sicurezza del sistema. Parallelamente, i Large Language Models (LLM) hanno rivoluzionato il campo dell'intelligenza artificiale, dimostrando capacità impressionanti nella comprensione e generazione di codice. Modelli come Code Llama, specializzati per compiti di programmazione, hanno raggiunto prestazioni notevoli su diversi linguaggi e task di sviluppo software. Questi modelli possono interpretare richieste in linguaggio naturale e produrre codice funzionale, abbassando significativamente la barriera tecnica per l'utilizzo di tecnologie complesse.

La generazione automatica di codice tramite LLM trova applicazione in diversi contesti: prototipazione rapida, automazione di task ripetitivi, assistenza allo sviluppo e democratizzazione dell'accesso a tecnologie specializzate. Nel caso specifico di eBPF, un sistema di generazione automatica permetterebbe a sviluppatori, amministratori di sistema e ricercatori di sfruttare le potenzialità della tecnologia senza necessariamente possedere conoscenze approfondite.

Il punto di partenza di questa tesi è **Kgent** (Kernel Extensions Large Language Model Agent), noto anche come **KEN**, un agente intelligente recentemente proposto da Zheng et al. [13] e disponibile come progetto open-source [1]. KEN affronta il problema della complessità nello sviluppo di programmi eBPF, consentendo di generare codice a partire da richieste in linguaggio naturale mediante l'integrazione di modelli linguistici di grandi dimensioni (LLM), tecniche di comprensione del codice e di esecuzione simbolica.

Il framework KEN, nella sua implementazione originale, si basa su *LangChain* e sul servizio *ChatOpenAI*, con conseguente dipendenza da API proprietarie e a pagamento. Questa scelta introduce limiti pratici di accessibilità e scalabilità, rendendo difficile la riproduzione del sistema in ambienti privi di risorse economiche dedicate o di accesso alle API.

Questa tesi si propone di adattare il framework originale in un contesto differente.

In particolare, il lavoro si concentra sull'esplorazione di modelli alternativi rispetto all'approccio originario, valutando l'integrazione di diverse soluzioni open-source ed eseguibili in locale. L'obiettivo è verificare la riproducibilità del sistema e analizzare la possibilità di sostituire modelli proprietari con alternative più accessibili, in grado di garantire maggiore flessibilità.

In questa prospettiva, la tesi si configura come un lavoro di validazione e adattamento tecnologico, volto da un lato a dimostrare la capacità del framework KEN di integrarsi con differenti famiglie di modelli linguistici, e dall'altro a valutare comparativamente l'efficacia di tali modelli. L'analisi prende in considerazione aspetti come la qualità del codice generato, la coerenza con le richieste in linguaggio naturale e il grado di aderenza ai prompt, al fine di evidenziare punti di forza e limiti delle diverse soluzioni adottate.

La tesi si articola in quattro capitoli: il Capitolo 1 introduce eBPF, bpftrace e il framework KEN; il Capitolo 2 descrive la metodologia di valutazione e i modelli testati; il Capitolo 3 presenta i risultati sperimentali; il Capitolo 4 conclude con sintesi e prospettive future.

Capitolo 1

Background

Questo capitolo fornisce le fondamenta teoriche e tecnologiche necessarie per comprendere il lavoro svolto. La prima sezione presenta una panoramica completa dei modelli linguistici di grandi dimensioni, dalla loro architettura di base fino ai modelli specializzati per la generazione di codice. La seconda sezione introduce eBPF, analizzandone l'architettura, le capacità e i principali casi d'uso. Infine, viene discusso il concetto di agenti intelligenti e il loro ruolo nell'automazione dello sviluppo software.

1.1 Large Language Models

1.1.1 Dalle Origini agli LLM

La rappresentazione del linguaggio naturale in forma numerica ha costituito una sfida centrale per decenni nel campo dell'elaborazione del linguaggio naturale. I primi approcci si basavano su rappresentazioni simboliche o bag-of-words, incapaci di catturare relazioni semantiche tra termini. L'introduzione di **word embeddings** ha segnato un cambio di paradigma: parole con significati simili vengono mappate in regioni prossime di uno spazio vettoriale continuo.

Il modello **Word2Vec**, proposto da Mikolov et al. nel 2013 [6], ha dimostrato che semplici architetture neurali possono apprendere rappresentazioni dense attraverso task predittivi (CBOW e Skip-gram). Questi embedding catturano relazioni analogiche e permettono operazioni aritmetiche vettoriali con risultati semanticamente coerenti (es. $\vec{re} - \vec{uomo} + \vec{donna} \approx \vec{regina}$).

Nonostante il successo degli embedding statici, la loro limitazione principale risiede nell'assenza di contestualizzazione: ogni parola ha una singola rappresentazione indipendentemente dal contesto. Le **Recurrent Neural Networks** (RNN) hanno introdotto la capacità di processare sequenze mantenendo uno stato interno che si aggiorna iterativamente. Tuttavia, le RNN vanilla soffrono del problema del *vanishing gradient*, che impedisce l'apprendimento di dipendenze a lungo raggio.

Le **Long Short-Term Memory** (LSTM), introdotte da Hochreiter e Schmidhuber nel 1997 [4], hanno risolto in parte i limiti delle RNN grazie a meccanismi di memoria basati su gate, diventando per anni lo standard in NLP. Tuttavia, il processamento strettamente sequenziale ne ha limitato la scalabilità e la capacità di catturare dipendenze a lungo raggio. L'architettura **Transformer**, proposta nel 2017 [12], ha superato questi limiti eliminando la ricorrenza e introducendo il mec-

canismo di *self-attention*, che consente elaborazione parallela ed efficace di sequenze lunghe.

1.1.2 Architettura Transformer

L'architettura Transformer, introdotta da Vaswani et al. nel 2017 [12], ha rivoluzionato il campo del Natural Language Processing. A differenza delle architetture precedenti che processavano il testo parola per parola in sequenza, i Transformer possono analizzare tutte le parole di una frase contemporaneamente.

Il concetto chiave è il meccanismo di **attention**: quando il modello processa una parola, può "guardare" tutte le altre parole nella frase e decidere quali sono più rilevanti per comprendere il contesto. Ad esempio, nella frase *"Il gatto che viveva nella casa mangiò il pesce"*, quando il modello analizza la parola *"mangiò"*, può capire che *"gatto"* è il soggetto anche se ci sono molte parole in mezzo.

Questo meccanismo permette ai modelli di:

- comprendere relazioni tra parole lontane nella frase;
- processare il testo in modo più efficiente;
- scalare a dimensioni molto grandi (miliardi di parametri).

I componenti principali di un Transformer includono:

- **Multi-Head Attention**: Il modello usa diversi meccanismi di attention in parallelo. Ogni "testa" si specializza nel catturare pattern diversi, come relazioni sintattiche o semantiche.
- **Feed-Forward Networks**: Reti neurali che trasformano le informazioni processate dall'attention.
- **Positional Encoding**: Siccome il Transformer analizza tutte le parole insieme, ha bisogno di un sistema per capire l'ordine delle parole nella frase.
- **Layer Normalization**: Tecniche che rendono il training più stabile e permettono di costruire modelli molto profondi.

Scalabilità

La chiave del successo dei Transformer risiede nella loro scalabilità: la parallelizzazione del calcolo consente di sfruttare in modo efficiente l'hardware moderno (GPU, TPU). Inoltre, l'architettura mostra un comportamento di scaling regolare: aumentando il numero di parametri, la quantità di dati e la potenza di calcolo, le prestazioni migliorano in modo prevedibile secondo leggi di potenza (power laws), cioè relazioni matematiche che descrivono come l'aumento delle risorse porta a miglioramenti proporzionati nei risultati.

Questa proprietà ha motivato la costruzione di modelli sempre più grandi, da BERT (110M parametri) a GPT-3 (175B) [2] fino ai modelli attuali con centinaia di miliardi o trilioni di parametri. La capacità di scaling ha reso i Transformer l'architettura dominante non solo in NLP ma anche in computer vision, speech e altri domini.

1.1.3 Evoluzione dei Large Language Models

Prima Generazione: GPT e BERT

I primi LLM basati su Transformer hanno introdotto due paradigmi distinti: **GPT** (Generative Pre-trained Transformer), con architettura decoder-only e training autoregressivo, e **BERT** (Bidirectional Encoder Representations from Transformers), con architettura encoder-only e masked language modeling.

GPT utilizza un obiettivo di language modeling causale: predire il prossimo token data la sequenza precedente. Questo approccio si è rivelato particolarmente efficace per task generativi. BERT, invece, maschera casualmente token nell'input e addestra il modello a ricostruirli avendo accesso al contesto bidirezionale, eccellendo in task di comprensione.

Entrambi i modelli hanno dimostrato che il *pre-training* su grandi corpus seguita da *fine-tuning* su task specifici porta a performance superiori rispetto al training da zero. Questo paradigma denominato "transfer learning" è diventato standard nel deep learning moderno.

Con **GPT-2** e soprattutto **GPT-3** (175B parametri) [2], OpenAI ha mostrato come le **scaling laws** leghino in modo prevedibile le performance alla dimensione del modello, giustificando lo sviluppo di LLM sempre più grandi. GPT-3 ha inoltre introdotto l'*in-context learning*, dimostrando che modelli di scala sufficiente possono affrontare nuovi task senza bisogno di fine-tuning.

GPT-4 e Capacità Emergenti

GPT-4 rappresenta un ulteriore salto qualitativo, con architettura e dimensione non pubblicamente divulgate ma stimate nell'ordine dei trilioni di parametri. Il modello dimostra capacità avanzate in ragionamento, comprensione multimodale (testo e immagini) e generazione di codice.

Le **capacità emergenti** sono abilità che appaiono improvvisamente superando determinate soglie dimensionali, non presenti in modelli più piccoli. Esempi includono:

- **Chain-of-thought reasoning:** decomposizione di problemi complessi in passi intermedi espliciti, migliorando significativamente le performance su task che richiedono ragionamento multi-step;
- **Few-shot learning avanzato:** capacità di apprendere nuovi task da pochissimi esempi, adattandosi rapidamente a domini specializzati;
- **Instruction following:** comprensione ed esecuzione precisa di istruzioni complesse formulate in linguaggio naturale;
- **Meta-learning implicito:** abilità di "imparare a imparare" durante l'interazione, migliorando le performance su task correlati.

Queste capacità suggeriscono che lo scaling non produce solo miglioramenti quantitativi ma cambiamenti qualitativi nel comportamento del modello, aprendo nuove possibilità applicative.

1.1.4 LLM per la Generazione di Codice

Modelli Specializzati

Mentre i LLM generici mostrano buone capacità nella generazione di codice, i modelli specializzati offrono risultati migliori. **Codex**, sviluppato da OpenAI e alla base di GitHub Copilot, è stato addestrato su miliardi di righe di codice e ha ottenuto buone prestazioni su benchmark come HumanEval. **Code Llama** [8], rilasciato da Meta nel 2023, è una variante di Llama 2 ottimizzata per la programmazione, disponibile in diverse versioni dedicate a completion, Python e instruction-following. Anche **StarCoder** e **StarCoder2**, modelli open-source addestrati su dataset di codice con licenze permissive, hanno mostrato risultati competitivi. Infine, **DeepSeek Coder** utilizza tecniche avanzate per bilanciare qualità ed efficienza. Questi modelli dimostrano il potenziale dei LLM come strumenti di supporto nello sviluppo software.

Rigidità Sintattica e Semantica

La generazione di codice richiede maggiore rigore rispetto al testo naturale, poiché gli LLM devono rispettare sia la sintassi sia la semantica del linguaggio. Gli errori sintattici sono relativamente rari, grazie all'apprendimento dei pattern dai dati di training, mentre la correttezza semantica rimane più critica: i modelli possono introdurre funzioni inesistenti, generare bug logici, ignorare casi limite o produrre codice inefficiente. Nei linguaggi con strong typing molti errori vengono intercettati dal compilatore, ma in contesti complessi come eBPF la validazione rimane particolarmente difficile.

Applicazioni e Limiti

I code LLM trovano impiego in attività come completamento e traduzione di codice, generazione da linguaggio naturale, bug fixing, documentazione e test. Tuttavia, presentano limiti importanti: assenza di garanzie di correttezza, dipendenza dai dati di training, contesto limitato, output non deterministico e bias ereditati dai dataset. Nel caso di eBPF, tali problemi risultano amplificati a causa dei vincoli del verifier e dell'elevata sensibilità agli errori.

1.2 eBPF: Extended Berkeley Packet Filter

1.2.1 Storia ed Evoluzione

Il **Berkeley Packet Filter (BPF)** è introdotto da Steven McCanne e Van Jacobson nel 1992 [5] per permettere il filtraggio efficiente dei pacchetti di rete in *userspace*. L'architettura BPF definisce una semplice macchina virtuale con un set di istruzioni limitato, eseguibile nel kernel per evitare costosi *context switch*.

Per oltre vent'anni, BPF rimase una tecnologia di nicchia, utilizzata principalmente da tool di network debugging come tcpdump e Wireshark. Il design originale, seppur efficace per il suo scopo specifico, presentava limitazioni significative: set di istruzioni ridotto, assenza di strutture dati complesse.

Nel 2014 **Alexei Starovoitov** introdusse **eBPF** (inizialmente "extended BPF"), trasformando il vecchio BPF da semplice filtro di pacchetti a piattaforma general-purpose per l'esecuzione sicura di programmi nel kernel. Le principali novità riguardavano:

- un set di istruzioni ampliato,
- il supporto a *helper functions*,
- strutture dati persistenti (*BPF maps*),
- un *verifier* avanzato,
- la compilazione JIT per prestazioni native.

Questa evoluzione segnò un vero cambio di paradigma, estendendo eBPF da strumento di rete a infrastruttura versatile per osservabilità, sicurezza e programmazione del kernel [3].

1.2.2 Architettura e Componenti

I programmi **eBPF** sono piccoli frammenti di codice che vengono eseguiti direttamente nel kernel in risposta a determinati eventi. Vengono in genere scritti in un subset di C, compilati in bytecode tramite **Clang/LLVM** e caricati tramite la syscall `bpf()`. L'architettura di eBPF fornisce un set di registri e istruzioni abbastanza ricco da permettere logiche complesse, ma allo stesso tempo abbastanza restrittivo da garantirne la verificabilità e la sicurezza.

Un elemento centrale sono le **BPF maps**, strutture dati condivise che permettono di mantenere stato e comunicare tra kernel e spazio utente. Ne esistono diverse tipologie (hash map, array, ring buffer, ecc.), adatte a scenari differenti come il tracciamento di eventi o la raccolta di statistiche.

A supporto dei programmi, il kernel mette a disposizione un insieme di **helper functions**, che consentono operazioni non realizzabili direttamente in bytecode, come leggere informazioni di processo, accedere alle map o inviare eventi allo spazio utente. L'elenco di queste funzioni viene continuamente ampliato con le nuove versioni del kernel.

Il Verifier

Il **verifier** è il componente critico che garantisce la sicurezza di eBPF [3]. Prima di caricare un programma, il verifier esegue analisi statica esaustiva simulando tutte le possibili esecuzioni. L'obiettivo è garantire che il programma:

1. **Termini sempre:** non contenga loop infiniti o ricorsione illimitata;
2. **Non acceda memoria illegalmente:** tutti i dereference di puntatori siano validati;
3. **Non causi kernel panic:** operazioni pericolose (divisione per zero, NULL dereference) siano previste;
4. **Rispetti i limiti di complessità:** non ecceda il numero massimo di istruzioni (tipicamente 1M).

Il verifier traccia lo stato dei registri a ogni istruzione (tipi, range, proprietà) ed esplora tutti i possibili branch per garantire sicurezza.

1.2.3 Casi d'Uso Principali

Networking e Observability

eBPF ha avuto un impatto enorme sul networking e sull'osservabilità in Linux. Nel primo ambito consente di processare pacchetti già a livello driver, implementando meccanismi ad alte prestazioni per mitigazione DDoS, load balancing e firewalling, fino al controllo fine sui socket.

Per l'osservabilità, eBPF si è affermato come riferimento nelle distribuzioni Linux moderne grazie alla capacità di tracciare eventi kernel e applicativi con overhead minimo. Permette di monitorare funzioni, system calls e performance dell'intero sistema in tempo reale, abilitando strumenti come **bcc**, **bpfftrace** e **Pixie**, oggi ampiamente adottati in contesti cloud e containerizzati.

Un ulteriore ambito di applicazione è la **security**: eBPF consente policy personalizzate a livello kernel, filtraggio avanzato delle system call e rilevamento di anomalie in tempo reale.

1.2.4 Limiti e Complessità

L'adozione di eBPF è frenata dalla sua complessità: scrivere programmi richiede una buona conoscenza del kernel Linux e dei principi di system programming, e il debugging è reso difficile dal fatto che il codice gira in kernel space, dove gli errori possono avere conseguenze gravi. Strumenti di livello più alto come **bcc** o **bpfftrace** aiutano a ridurre la barriera d'ingresso, ma restano limitati per casi d'uso avanzati.

Un'altra difficoltà è rappresentata dal **verifier**, che controlla la sicurezza dei programmi prima dell'esecuzione. Pur essendo fondamentale, impone regole rigide e talvolta poco trasparenti, con messaggi di errore difficili da interpretare e tempi di verifica lunghi, costringendo spesso a strategie di tentativi ed errori.

Infine, lo sviluppo con eBPF richiede conoscenze approfondite del kernel e dei suoi sottosistemi, oltre alla consapevolezza che API e strutture possono cambiare tra versioni diverse. Soluzioni come BPF CO-RE e **libbpf** cercano di mitigare questi problemi di portabilità, ma la curva di apprendimento rimane comunque ripida.

1.3 Agenti Intelligenti

Un **agente intelligente** è un sistema che prende decisioni e agisce in modo autonomo per raggiungere un obiettivo [10]. In termini semplici, osserva l'ambiente in cui si trova, elabora le informazioni disponibili e sceglie quali azioni compiere. Gli agenti possono anche imparare dall'esperienza, migliorando progressivamente il proprio comportamento.

Negli ultimi anni, grazie ai **LLM**, gli agenti hanno assunto un ruolo nuovo e più potente. Un LLM può infatti fare da "cervello" dell'agente: non solo produce testo, ma può anche ragionare, pianificare e interagire con strumenti esterni come motori di ricerca, database o ambienti di sviluppo [14]. In questo modo, un agente non si

limita a rispondere a domande, ma è in grado di portare avanti attività complesse in autonomia.

Un esempio pratico sono gli **agenti per lo sviluppo software** [1], che possono generare codice, eseguire test, leggere documentazione e persino correggere errori. Questo rende la programmazione più accessibile, riducendo la barriera di ingresso a tecnologie complesse come **eBPF**, e permette agli sviluppatori di concentrarsi sulle idee invece che sui dettagli più tecnici.

Gli agenti intelligenti possono quindi essere visti come assistenti virtuali avanzati: strumenti che combinano comprensione del linguaggio naturale con capacità operative concrete, aprendo la strada a una nuova forma di automazione flessibile e adattiva.

Capitolo 2

Metodologia

Questo capitolo illustra il percorso di analisi e adattamento di Kgent: dalla descrizione della sua architettura originale, alle difficoltà incontrate nella replicazione, fino alle modifiche necessarie per sostituire i modelli proprietari con soluzioni open-source locali.

2.1 Struttura

2.1.1 Panoramica e Obiettivi del Progetto

Kgent [1], già noto come KEN, utilizza i LLM per tradurre richieste in linguaggio naturale in programmi eBPF funzionanti, con l'obiettivo di rendere più accessibile lo sviluppo di estensioni del kernel. Il sistema nasce per superare le barriere tecniche legate a conoscenze del kernel, vincoli del **verifier** e complessità di debugging, semplificando il processo per gli sviluppatori. Il flusso principale di Kgent può essere sintetizzato in quattro fasi: riceve in input una descrizione testuale dell'utente (ad esempio, "traccia tutte le connessioni TCP"), analizza l'intento per identificare i sottosistemi kernel coinvolti, genera il codice eBPF utilizzando un LLM guidato da esempi rilevanti e knowledge base specializzate, e infine esegue il programma dopo averlo validato. Questo processo integra meccanismi di verifica, retry e feedback che permettono di correggere errori e migliorare progressivamente la qualità del codice generato.

2.1.2 Architettura del Sistema

L'architettura di Kgent si articola in cinque componenti principali che collaborano per trasformare linguaggio naturale in codice eBPF eseguibile. Ogni componente svolge un ruolo specifico nel workflow complessivo, e la loro integrazione costituisce il cuore del sistema.

Elaborazione dell'Input

Il primo componente si occupa di processare la richiesta dell'utente. Un parser analizza il testo in linguaggio naturale per identificare gli elementi chiave: quali eventi del kernel tracciare, quali processi filtrare, quali metriche raccogliere. Il sistema di riconoscimento dell'intento classifica la richiesta in categorie (networking, filesystem, memory, syscall) e determina il livello di complessità richiesto. Questa fase è cruciale

perché condiziona le scelte successive, in particolare la selezione degli esempi e dei punti di aggancio al kernel.

Knowledge Base

Il cuore della conoscenza di Kgent risiede in tre elementi distinti ma complementari. Il primo sono i **database vettoriali**, implementati utilizzando FAISS (Facebook AI Similarity Search), una libreria sviluppata da Meta per effettuare ricerche di similarità ad alta efficienza su grandi insiemi di vettori. Il sistema mantiene due database separati: uno contiene esempi di programmi eBPF basati su libbpf scritti in C, l'altro raccoglie programmi bpfttrace che utilizzano un linguaggio di scripting di livello più alto. Questi esempi vengono convertiti in embedding utilizzando l'API di OpenAI e indicizzati in FAISS, così da permettere ricerche rapide basate sulla similarità semantica con la query dell'utente.

Il secondo elemento è il **ricercatore di punti di aggancio** (hook finder), che determina quali tracepoint, kprobe o altri meccanismi di tracciamento utilizzare. Il sistema combina pattern matching basato su regex con l'esecuzione diretta del comando `bpfttrace -l` per identificare i punti di aggancio disponibili nel kernel corrente. Questa fase è essenziale perché i programmi eBPF dipendono strettamente dai punti di strumentazione specifici del kernel in esecuzione.

Il terzo elemento sono le **specifiche SMT** (Satisfiability Modulo Theories), che forniscono definizioni formali del comportamento delle helper functions eBPF e delle kprobe. Queste specifiche vengono utilizzate dal componente di verifica per validare la correttezza logica dei programmi generati, andando oltre i controlli sintattici del verifier eBPF standard.

Catena di Generazione LLM

Questo componente orchestra l'interazione con il Large Language Model per produrre il codice eBPF. Il sistema utilizza una strategia di **few-shot learning**: costruisce un prompt che include sia esempi semplici predefiniti sia esempi complessi recuperati dal database vettoriale in base alla similarità con la richiesta dell'utente. Gli esempi fungono da modelli che guidano l'LLM verso la generazione di codice conforme alle best practices. L'ingegneria dei prompt segue convenzioni precise: utilizza tag di sistema per strutturare le istruzioni, specifica requisiti di formattazione stringenti e include direttive esplicite per evitare spiegazioni testuali e concentrarsi esclusivamente sulla generazione di codice pulito e funzionale.

Verifica e Miglioramento

Una volta generato il programma, il sistema attiva una fase di validazione multi-livello. Il componente di **esecuzione simbolica** utilizza il solver SMT Z3 per verificare la correttezza del programma rispetto alle specifiche formali. Questa analisi può identificare errori logici sottili che il verifier eBPF standard non rileva, come accessi a campi di strutture non validi per determinati contesti o uso improprio di helper functions. Se il programma fallisce durante la compilazione o l'esecuzione, si attiva un **meccanismo di retry** che cattura il messaggio di errore, lo incorpora nel prompt originale e chiede al LLM di correggere il problema. Questo processo si

ripete fino a tre volte, permettendo al sistema di risolvere errori sintattici, di contesto o logici attraverso iterazioni successive. Il sistema prevede anche un **loop di feedback umano** opzionale, che permette a esperti di fornire suggerimenti per migliorare la qualità del programma generato. Questo meccanismo è particolarmente utile per casi complessi dove l'automazione completa potrebbe non essere sufficiente.

Esecuzione e Workflow

Il componente finale di Kgent gestisce la compilazione ed esecuzione dei programmi: per **bpftrace** usa direttamente l'interprete, mentre per **libbpf** si affida alla toolchain LLVM. I risultati vengono poi restituiti in formato JSON strutturato con output, errori e metriche di performance.

Un tipico workflow segue questi passi: l'utente formula una richiesta in linguaggio naturale (es. monitorare i file aperti da nginx), il sistema ne analizza l'intento e recupera esempi simili dal database vettoriale, identifica i tracepoint pertinenti, costruisce un prompt arricchito e lo invia al LLM. Il codice generato viene verificato (anche tramite SMT), corretto in caso di errori e infine eseguito. L'output strutturato viene mostrato all'utente.

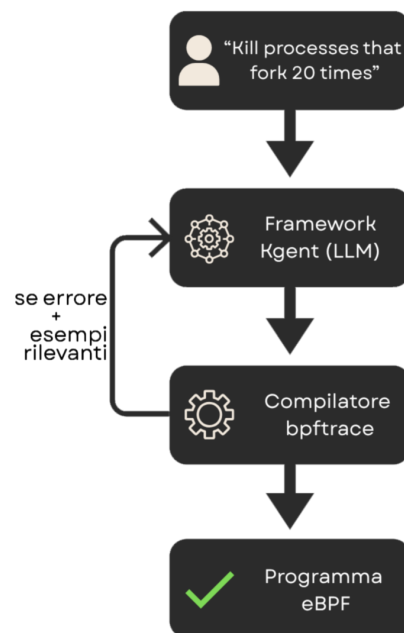


Figura 2.1: Workflow di generazione e verifica automatica dei programmi eBPF in Kgent.

Questa pipeline integrata — analisi dell'intento, recupero di esempi, generazione LLM, verifica e retry — rappresenta il nucleo innovativo di Kgent, garantendo maggiore affidabilità rispetto all'uso diretto di modelli linguistici generici.

2.2 Problematiche nella Replicazione

2.2.1 Scoperta dei Vincoli Operativi

Primo Tentativo di Esecuzione

La fase di replica è iniziata con il tentativo di eseguire il codice originale del repository. Il README del progetto indicava la necessità di una chiave API OpenAI, ottenibile tramite registrazione al servizio. Utilizzando un account personale OpenAI, è stato possibile configurare l'ambiente di esecuzione:

```
export OPENAI_API_KEY="sk-..."
```

Il sistema è stato avviato con successo e ha iniziato a processare le prime richieste. Tuttavia, dopo pochi secondi di esecuzione, il programma si è interrotto con un errore nei log:

```
Error: You reached the limit, OUT OF TOKENS  
Rate limit exceeded for API requests
```

Analisi del Problema

L'analisi dei log ha rivelato che il problema non era legato a errori nel codice, ma a limitazioni del modello di billing di OpenAI. Ogni account gratuito o con credito limitato ha un **threshold di token** massimo utilizzabile, oltre il quale le richieste API vengono bloccate.

Il framework Kgnt, per la sua natura di sistema multi-agente con meccanismi di retry, few-shot learning e verifica iterativa, genera un volume significativo di richieste API anche per una singola query utente. Il flusso tipico include:

1. generazione embedding per ricerca nel vector database;
2. generazione del codice eBPF (prima chiamata LLM);
3. eventuali retry in caso di errore di compilazione (fino a 3 tentativi);
4. verifica SMT e rigenerazione (ulteriori chiamate).

Questo comporta che anche un singolo test case possa consumare diverse migliaia di token, rendendo il threshold degli account gratuiti insufficiente per eseguire anche solo le operazioni di base del sistema.

2.2.2 Decisione di Migrazione a Modelli Locali

Utilizzo di Modelli gratuiti

Il repository menzionava nel README il supporto per modelli alternativi (GPT-3.5-turbo, CodeLlama via DeepInfra). Tuttavia, l'analisi del codice ha evidenziato che:

- le funzioni utilizzavano `ChatOpenAI` di `LangChain` in modo hard-coded;
- gli embedding erano generati tramite `OpenAIEmbeddings`;

- anche GPT-3.5-turbo, pur meno costoso, richiedeva billing attivo;
- i servizi di terze parti (DeepInfra) presentavano limitazioni analoghe.

La necessità di migrare verso modelli locali è emersa come conseguenza diretta delle limitazioni imposte dal sistema di billing di OpenAI. L'elevato numero di chiamate generate dal framework Kgent — dovuto alla sua architettura multi-agente, ai meccanismi di retry e ai processi di verifica iterativa — rendeva impraticabile l'utilizzo di chiavi API gratuite o con credito limitato, poiché il consumo di token superava rapidamente le soglie consentite.

Sebbene il repository dichiarasse il supporto a modelli alternativi come **CodeLlama** o **DeepSeek**, l'analisi del codice ha evidenziato una forte dipendenza da componenti proprietarie OpenAI, in particolare dalle classi **ChatOpenAI** e **OpenAIEmbeddings**. Inoltre, anche i modelli "economici" o le soluzioni di terze parti richiedevano comunque un account con billing attivo, vanificando l'obiettivo di ottenere un'infrastruttura completamente gratuita e autonoma.

Per questi motivi, la migrazione verso LLM open-source eseguibili in locale si è configurata come l'unica soluzione sostenibile, consentendo di eliminare ogni dipendenza da servizi cloud a pagamento, mantenere pieno controllo sulle risorse computazionali e garantire la riproducibilità del sistema in contesti accademici o di ricerca.

2.3 Strategia di Migrazione ai Modelli Locali

2.3.1 Percorso di Adozione

La transizione verso modelli locali è avvenuta in più fasi. Il primo tentativo, basato su **transformers** di HuggingFace con **CodeLlama-7B**, ha evidenziato complessità nella gestione della GPU e dei parametri di inferenza. L'alternativa con **LLaMa.cpp**, pur più leggera, si è rivelata troppo lenta su CPU e instabile con supporto CUDA, rendendola impraticabile.

2.3.2 Adozione di Ollama

La soluzione ottimale è stata l'adozione di **Ollama** [7], piattaforma che consente l'esecuzione locale di LLM tramite un'API REST standard. Offre installazione immediata, gestione automatica della GPU e dei modelli, oltre a uno switching rapido tra diverse architetture senza modifiche al codice.

```
curl https://ollama.ai/install.sh | sh
ollama pull codellama:7b-instruct
ollama serve
```

Listing 2.1: Installazione e configurazione di Ollama

Come baseline è stato scelto **CodeLlama-7B-Instruct**, per il buon equilibrio tra dimensioni, capacità di instruction-following e compatibilità con l'hardware disponibile (4–5 GB di VRAM).

2.3.3 Infrastruttura Multi-Modello

Per la successiva valutazione comparativa (Capitolo 3) sono stati installati diversi modelli, inclusi CodeLlama (7B–34B), DeepSeek Coder (6.7B–16B) e WizardCoder (13B). La configurazione multi-modello consente di eseguire test variando solo il parametro del modello nelle chiamate API. I moduli principali (`chain.py`, `gpttrace.py`, `verifier.py`) sono stati aggiornati mantenendo inalterata la logica generale del sistema.

2.3.4 Migrazione degli Embeddings

Il sistema di embeddings, inizialmente basato su `OpenAIEmbeddings`, è stato migrato a `HuggingFaceEmbeddings` con il modello `sentence-transformers/all-MiniLM-L6-v2`, richiedendo il re-embedding completo del database vettoriale FAISS e completando la transizione verso un'infrastruttura totalmente locale e open-source.

2.4 Adattamenti Implementativi

2.4.1 Architettura Distribuita

Configurazione Infrastrutturale

L'implementazione della soluzione ha richiesto la configurazione di un'architettura distribuita su due macchine virtuali distinte, ospitate all'interno dell'infrastruttura **HPC4AI** [11]. Ciascuna macchina è stata predisposta con ruoli specifici, determinati da vincoli operativi e di sicurezza, come illustrato in Figura 2.2.

La macchina **alpha-v100**, dotata di **GPU NVIDIA Tesla V100**, è stata dedicata all'esecuzione di **Ollama** [7] e all'inferenza dei modelli linguistici di grandi dimensioni. Dal punto di vista hardware, la VM dispone di **16 vCPU Intel Xeon (Cascadelake)**, **256 GB di RAM** e uno spazio di archiviazione complessivo di circa **9.8 TB** montato in `/mnt/home_new`. Il sistema operativo installato è **Ubuntu 24.04 LTS** con kernel `6.8.0-60-generic`.

Parallelamente, la macchina **alpha-relay**, utilizzata come ambiente di sviluppo e testing per l'esecuzione dei programmi eBPF generati, è equipaggiata con **8 vCPU Intel Xeon (Cascadelake)**, **16 GB di RAM** e un disco da **50 GB**. Esegue **Ubuntu 22.04 LTS** con kernel `5.15.0-113-generic` e l'utente dispone dei privilegi `sudo` necessari alle operazioni di compilazione e caricamento dei moduli nel kernel Linux.

La macchina **alpha-v100**, dotata di **GPU NVIDIA Tesla V100**, è stata dedicata all'esecuzione di **Ollama** [7] e all'inferenza dei modelli linguistici di grandi dimensioni. Dal punto di vista hardware, la VM dispone di **16 vCPU Intel Xeon (Cascadelake)**, **256 GB di RAM** e uno spazio di archiviazione complessivo di circa **9.8 TB** montato in `/mnt/home_new`. Il sistema operativo installato è **Ubuntu 24.04 LTS** con kernel `6.8.0-60-generic`.

Parallelamente, la macchina **alpha-relay**, utilizzata come ambiente di sviluppo e testing per l'esecuzione dei programmi eBPF generati, è equipaggiata con **8 vCPU Intel Xeon (Cascadelake)**, **16 GB di RAM** e un disco da **50 GB**. Esegue **Ubuntu 22.04 LTS** con kernel `5.15.0-113-generic` e l'utente dispone dei pri-

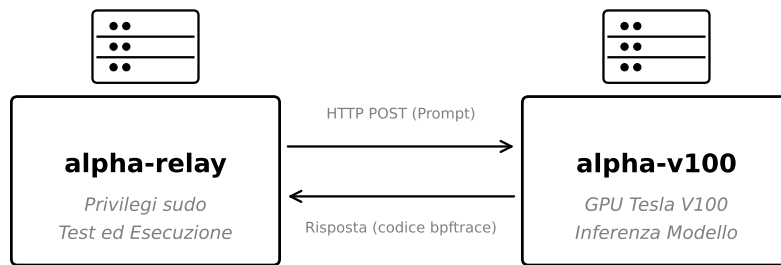


Figura 2.2: Architettura distribuita: comunicazione tra le macchine virtuali alpha-relay e alpha-v100

vilegi **sudo** necessari alle operazioni di compilazione e caricamento dei moduli nel kernel Linux.

Questa separazione nasce da necessità tecniche precise. L'esecuzione di programmi eBPF richiede privilegi di amministratore per caricare codice direttamente nel kernel Linux, operazione che presenta rischi potenziali per la stabilità del sistema. La macchina **alpha-relay** è stata fornita specificamente per questo scopo, con permessi sudo limitati all'esecuzione di bpfttrace e comandi correlati, mantenendo il workspace di sviluppo isolato dalla macchina con GPU. D'altra parte, l'inferenza dei modelli linguistici richiede accelerazione hardware per ottenere tempi di risposta accettabili: un modello come CodeLlama-7B, senza GPU, richiederebbe decine di secondi per generare anche programmi semplici, rendendo impraticabile il sistema di retry iterativo.

Installare Ollama su **alpha-v100** permette quindi di sfruttare la Tesla V100 per la generazione del codice, riducendo i tempi di inferenza a pochi secondi, mentre il testing e il debugging dei programmi generati avvengono su **alpha-relay** in un ambiente controllato con privilegi appropriati.

Comunicazione tra Macchine Virtuali

Per collegare le due macchine è stato implementato un meccanismo basato su richieste HTTP. Il modulo `ollama_utils.py` è stato creato come interfaccia centralizzata che invia richieste POST all'endpoint Ollama ospitato su **alpha-v100**. La funzione principale `query_ollama()`, mostrata nel Listing 2.2, incapsula la logica di comunicazione remota, gestendo sia l'invio della richiesta che la ricezione della risposta.

L'indirizzo IP corrisponde alla macchina **alpha-v100** nella rete interna del centro HPC4AI. La funzione implementata incapsula tre aspetti critici: gestisce la comunicazione HTTP cross-machine, aggiunge prompt engineering specifico per guidare la generazione di codice bpfttrace valido, e implementa error handling per gestire timeout di rete o errori di risposta.

```

def query_ollama(prompt, max_tokens=1000):
    model = "codellama:7b"

    bpftrace_prompt = f"""Generate a complete bpftrace program for: {prompt}
Requirements:
- Use proper bpftrace syntax
- Include BEGIN and END blocks
- Use correct tracepoints/kprobes
- For interrupts use: tracepoint:irq:irq_handler_entry
- For processes use: tracepoint:sched:sched_process_fork
- For syscalls use: tracepoint:syscalls:sys_enter_*
Write ONLY the bpftrace program (no explanations):"""

    payload = {
        "model": model,
        "prompt": bpftrace_prompt,
        "stream": False,
        "options": {"temperature": 0.1, "max_tokens": max_tokens}
    }

    response = requests.post(
        "http://111.111.111:1111/api/generate",
        json=payload
    )

    if response.status_code == 200:
        return response.json().get("response", "")
    return None

```

Listing 2.2: Funzione per interrogare Ollama da remoto

Il prompt engineering integrato nella funzione è stato raffinato iterativamente durante lo sviluppo. Le istruzioni esplicite sui pattern comuni (tracepoint per interrupts, syscall, processi) aiutano il modello a generare codice più aderente alle convenzioni bpftrace, riducendo errori sintattici comuni. L'enfasi su "Write ONLY the bpftrace program" cerca di limitare la tendenza del modello a generare spiegazioni testuali insieme al codice, problema ricorrente con CodeLlama.

Funzione Specializzata per il Verifier

Oltre alla funzione standard, è stata implementata una variante dedicata al verifier SMT. La funzione `query_ollama_verifier()`, mostrata nel Listing 2.3, differisce dalla precedente per un aspetto fondamentale: non applica alcun prompt engineering aggiuntivo, passando il prompt ricevuto direttamente al modello senza modifiche.

```

def query_ollama_verifier(prompt, max_tokens=1000):
    model = "codellama:7b"

    payload = {
        "model": model,
        "prompt": prompt,
        "stream": False,
        "options": {"temperature": 0.1, "max_tokens": max_tokens}
    }

    response = requests.post(
        "http://111.111.111:1111/api/generate",
        json=payload
    )

    if response.status_code == 200:
        return response.json().get("response", "")
    return None

```

Listing 2.3: Funzione specializzata per il verifier SMT

La distinzione tra le due funzioni è motivata dal contesto d'uso. Quando il verifier chiama il modello durante un retry, il prompt contiene già informazioni altamente strutturate: il programma precedentemente generato, il messaggio di errore completo del compilatore o del verifier eBPF, indicazioni sui vincoli violati e contesto specifico del fallimento. Aggiungere ulteriore prompt engineering a questo punto rischierebbe di diluire le informazioni critiche contenute nei messaggi di errore, che spesso includono dettagli tecnici precisi (numeri di linea, simboli non risolti, violazioni di tipo). La funzione `query_ollama_verifier()` preserva quindi l'integrità del prompt costruito dal meccanismo di retry, permettendo al modello di focalizzarsi esclusivamente sull'errore specifico da correggere.

Workflow Completo

Il flusso operativo attraversa le due macchine in modo trasparente all'utente (Figura 2.2). Una richiesta inviata tramite l'API Flask su **alpha-relay** viene processata dal sistema che, internamente, chiama `query_ollama()` per inviare una richiesta HTTP a **alpha-v100**. Ollama esegue l'inferenza sfruttando la GPU Tesla V100 e restituisce il codice eBPF generato via HTTP ad **alpha-relay**. A questo punto il verifier compila ed esegue il programma utilizzando i privilegi `sudo` disponibili sulla macchina relay. Se si verificano errori di compilazione o esecuzione, il sistema di retry costruisce un nuovo prompt contenente l'errore e chiama `query_ollama_verifier()` per ottenere una versione corretta del programma. Questo ciclo può ripetersi fino a tre volte prima di restituire l'ultimo tentativo all'utente.

Dunque **alpha-v100** esegue l'inferenza del modello su GPU, mentre **alpha-relay** si occupa dell'esecuzione sicura e del testing del codice eBPF nel kernel.

2.4.2 Vantaggi del Modulo Centralizzato

La creazione di `ollama_utils.py` come punto di accesso unico a Ollama offre benefici architetturali significativi. Modificare il modello utilizzato richiede cambiare un solo parametro in un unico file, anziché intervenire su molteplici punti del codebase. Questa centralizzazione garantisce inoltre consistenza nei parametri di inferenza: temperatura, `max_tokens` e altre opzioni rimangono uniformi attraverso tutte le chiamate, eliminando comportamenti inconsistenti dovuti a configurazioni divergenti. Il debugging risulta semplificato grazie alla possibilità di aggiungere logging centralizzato che traccia tutte le interazioni con Ollama, facilitando l'identificazione di problemi di rete, timeout o errori di parsing delle risposte.

Tutti i file modificati del repository importano questo modulo, garantendo un'interfaccia uniforme verso Ollama. I file `chain.py`, `gpttrace.py`, `verifier.py`, `z3_conditions_for_ebpf.py` e `index.py` utilizzano le due funzioni esposte senza preoccuparsi dei dettagli implementativi della comunicazione HTTP o della struttura dei payload JSON.

2.4.3 Modifiche ai Componenti Core

File `chain.py`

Le funzioni principali di generazione codice sono state riscritte per utilizzare il modulo centralizzato. La funzione `run_gpt_for_bpftrace_progs()`, originariamente basata su `ChatOpenAI` e le catene `LangChain`, è stata semplificata a una chiamata diretta a `query_ollama` seguita dal parsing della risposta. Analogamente, `run_gpt_for_libbpf_progs()` è stata adattata per programmi eBPF in C, mantenendo la stessa struttura ma con prompt engineering specifico per il linguaggio C e le convenzioni `libbpf`. Le funzioni ausiliarie come `ask_gpt_for_question()` e `run_gpt_for_bpftrace_hooks()` hanno subito trasformazioni simili, sostituendo le dipendenze da API proprietarie con chiamate al modulo locale.

Una modifica importante riguarda la funzione `extract_code_blocks()`, estesa per gestire output non strutturato tipico di `CodeLlama`. Il modello tende a generare commenti esplicativi, frasi introduttive come “Here’s a bpftrace program that...” e markdown formatting (backticks tripli) che devono essere filtrati per estrarre solo il codice eseguibile. La funzione implementa una logica multi-step: rimozione dei backticks, filtraggio delle linee che contengono solo commenti o testo esplicativo, e rilevamento automatico dell’inizio effettivo del programma bpftrace cercando pattern come `kprobe:`, `tracepoint:` o `BEGIN`. Questo parsing robusto compensa le imperfezioni del modello, che nonostante le istruzioni esplicite nel prompt continua occasionalmente a generare output verboso.

La funzione `run_3trails()` per la gestione dei retry ha ricevuto una modifica critica nella condizione di successo. Originariamente, la funzione considerava completato il task solo se `returncode == 0`. Tuttavia, durante il testing è emerso che la versione recente di bpftrace genera errori `Could not resolve symbol: BEGIN_trigger` anche per programmi sintatticamente corretti, dovuti a incompatibilità con il trigger `BEGIN` deprecato. Per evitare retry inutili su errori ambientali non risolvibili modificando il codice, la condizione è stata estesa per considerare successo anche i casi con `returncode == 255` quando l’errore contiene “Could not resolve symbol” ma non “syntax error”.

Questo distingue errori di ambiente da errori effettivi di sintassi o logica del programma.

File `gpttrace.py`

Per mantenere la compatibilità con l’architettura agent-based di `LangChain`, utilizzata per orchestrare tool e memory nelle conversazioni multi-turn, è stata creata una classe wrapper `CodeLlamaLLM` che estende la classe base LLM di `LangChain`.

Questa classe implementa il metodo `_call()` che internamente invoca `ollama_utils.query_ollama()`, traducendo l’interfaccia di Ollama in quella attesa dal framework `LangChain`. In questo modo, gli agent esistenti continuano a funzionare senza modifiche strutturali, semplicemente istanziando `CodeLlamaLLM` al posto di `ChatOpenAI`. Le funzioni di ricerca nel vector database,

`get_top_n_example_from_bpftrace_vec_db()` e

`get_top_n_example_from_libbpf_vec_db()`,

aggiornate con `HuggingFaceEmbeddings` con il modello `sentence-transformers/all-MiniLM-L6-v2`.

File verifier.py

Le funzioni di retry del verifier SMT, `retry_generate_bpftime_program_for_compile()` e `retry_generate_bpftime_program_for_sat()`, sono state aggiornate per utilizzare `ollama_utils.query_ollama_verifier()` quando il model flag è impostato su "ollama". Queste funzioni costruiscono prompt complessi che includono il programma fallito, l'errore completo del compilatore o del verifier, e istruzioni specifiche su come correggere il problema. L'uso della funzione specializzata garantisce che questo contesto ricco venga preservato senza alterazioni.

File z3_conditions_for_ebpf.py

Questo file gestisce la generazione automatica delle condizioni Z3 per helper functions eBPF e kprobe, utilizzate dal verifier per validazione formale. La funzione `generate_response()` è stata modificata per accettare un parametro `llm_backend` che di default utilizza "ollama", invocando `query_ollama()` per generare specifiche SMT2 in formato JSON a partire dalle definizioni delle funzioni helper. Le funzioni `generate_libbpf_once()` e `generate_bpftime_once()` iterano sulle definizioni disponibili, generano le condizioni pre/post per ciascuna e salvano i risultati nel database di specifiche utilizzato successivamente dal verifier. Questa migrazione completa l'eliminazione delle dipendenze da OpenAI anche nella fase di preparazione delle knowledge base del sistema.

File index.py

L'API Flask è stata aggiornata per permettere la scelta dinamica del modello direttamente dal campo "model" nel payload JSON della richiesta. A seconda del valore inserito (ad esempio `code-llama`, `gpt-4`, ecc.), viene chiamata la funzione corrispondente:

- `run_ollama_for_bpftime_progs()` se si usa un modello locale tramite Ollama;
- `run_gpt_for_bpftime_progs()` per mantenere compatibilità con l'API OpenAI.

Questa modifica rende il backend flessibile, consentendo al frontend di specificare il modello da testare senza modificare il codice del server.

2.4.4 Downgrade di bpftime per Compatibilità

Durante la fase di testing dei programmi generati è emerso un problema critico di compatibilità con la versione di bpftime installata di default su **alpha-relay**. Molti programmi, pur essendo sintatticamente corretti e semanticamente validi, fallivano in esecuzione con l'errore:

```
ERROR: Could not resolve symbol: /proc/self/exe:BEGIN_trigger
```

Listing 2.4: Errore di incompatibilità con il trigger BEGIN

Questo errore appariva sistematicamente per programmi che utilizzavano il costrutto `BEGIN`, uno dei pattern più comuni in bpftime per inizializzazione di variabili globali o stampa di header prima dell'inizio del tracing vero e proprio. L'analisi del

problema ha rivelato che il trigger **BEGIN** è stato modificato nelle versioni recenti di bpftrace (post v0.17), introducendo una nuova implementazione interna che richiede simboli diversi rispetto alle versioni precedenti.

Il problema era particolarmente insidioso perché non si trattava di errori nel codice generato dal modello, ma di incompatibilità tra la sintassi utilizzata negli esempi del vector database (raccolti con versioni precedenti di bpftrace) e l'interprete installato sul sistema. Questo creava una situazione paradossale: il modello generava codice seguendo correttamente gli esempi forniti nel prompt, ma il codice risultante non era eseguibile sull'ambiente target. Il sistema di retry aggravava il problema, sprecando tentativi nel tentativo di "correggere" un codice già corretto.

La soluzione adottata è stata il downgrade di bpftrace alla versione v0.16.0, l'ultima release stabile che mantiene piena compatibilità con la sintassi legacy del trigger **BEGIN**. Il processo ha richiesto la ricompilazione completa di bpftrace dal codice sorgente:

```
git clone https://github.com/iovisor/bpftrace
cd bpftrace
git checkout v0.16.0
```

Listing 2.5: Downgrade di bpftrace alla versione 0.16.0

Il processo di build richiede diverse dipendenze (LLVM, Clang, libelf, libbpf) già presenti su **alpha-relay** per supportare lo sviluppo eBPF, rendendo la compilazione relativamente lineare.

Dopo il downgrade, i programmi contenenti **BEGIN** hanno iniziato a eseguire correttamente. Tuttavia, è emerso che alcuni errori **BEGIN_trigger** persistevano sporadicamente anche con la versione corretta, probabilmente dovuti a race condition o problemi nell'strumentazione dinamica del kernel. Per questo motivo, la funzione `run_3trails()` è stata modificata per distinguere errori di risoluzione simboli da errori sintattici veri, evitando retry inutili quando il fallimento è chiaramente ambientale:

```
if data["returncode"] == 0 or \
  (data["returncode"] == 255 and
   "Could not resolve symbol" in str(data["stderr"]) and
   "syntax error" not in str(data["stderr"])):
  return res
```

Listing 2.6: Gestione degli errori ambientali nella funzione di retry

Questa logica permette al sistema di considerare "successo tecnico" programmi che falliscono per problemi di ambiente ma sono sintatticamente e semanticamente corretti, distinguendoli da fallimenti dovuti a errori effettivi nel codice generato dal modello.

Capitolo 3

Risultati

3.1 Setup

3.1.1 Dataset di Test

La valutazione è stata condotta su un dataset di 40 test case rappresentativi delle principali categorie di utilizzo di eBPF. I test sono stati progettati per coprire un ampio spettro di complessità e funzionalità. Ogni test case è formulato come una descrizione in linguaggio naturale del comportamento desiderato. Esempi rappresentativi includono:

- “Monitor the rate of specific hardware interrupts and logs the interrupt sources”;
- “Trace process forking events and logs the parent and child process IDs and command names”;
- “Capture TCP SYN packets and logs the source and destination IP addresses and ports”;
- “Count LLC cache misses by process name”;
- “Trace directory entry cache (dcache) lookups by process.”

Il dataset è stato progettato per testare non solo la capacità di generare sintassi corretta, ma anche la comprensione semantica delle richieste, la scelta appropriata di hook points (tracepoint vs kprobe), e la corretta gestione di strutture dati kernel. I test sono stati inoltre suddivisi in cinque categorie principali: *Tracing*, *Syscalls*, *Networking*, *Filesystem* e *Advanced*, al fine di valutare le performance dei modelli in diversi ambiti funzionali di eBPF.

3.1.2 Modelli Valutati

Sono stati testati cinque modelli linguistici open-source di dimensioni variabili, appartenenti a due famiglie architetture distinte:

CodeLlama [9] è una famiglia di modelli specializzati per la generazione di codice, addestrati su dataset ricchi di codice sorgente in molteplici linguaggi. La variante “Instruct” è stata ulteriormente fine-tuned per seguire istruzioni in linguaggio naturale e generare codice conforme alle specifiche fornite.

DeepSeek-Coder-v2 [15] rappresenta una famiglia più recente, ottimizzato specificamente per task di code generation e reasoning su codice.

Modello	Parametri	Famiglia	Variante	Dimensione
CodeLlama-7B-Instruct	7 Billion	Meta LLaMA	Instruct	3.8 GB
CodeLlama-13B-Instruct	13 Billion	Meta LLaMA	Instruct	7.4 GB
CodeLlama-70B-Instruct	70 Billion	Meta LLaMA	Instruct	48 GB
DeepSeek-Coder-v2-16B	16 Billion	DeepSeek-v2	-	8.9 GB
DeepSeek-Coder-v2-236B	236 Billion	DeepSeek-v2	-	132 GB

Modelli LLM valutati nella sperimentazione

3.1.3 Metodologia di Valutazione

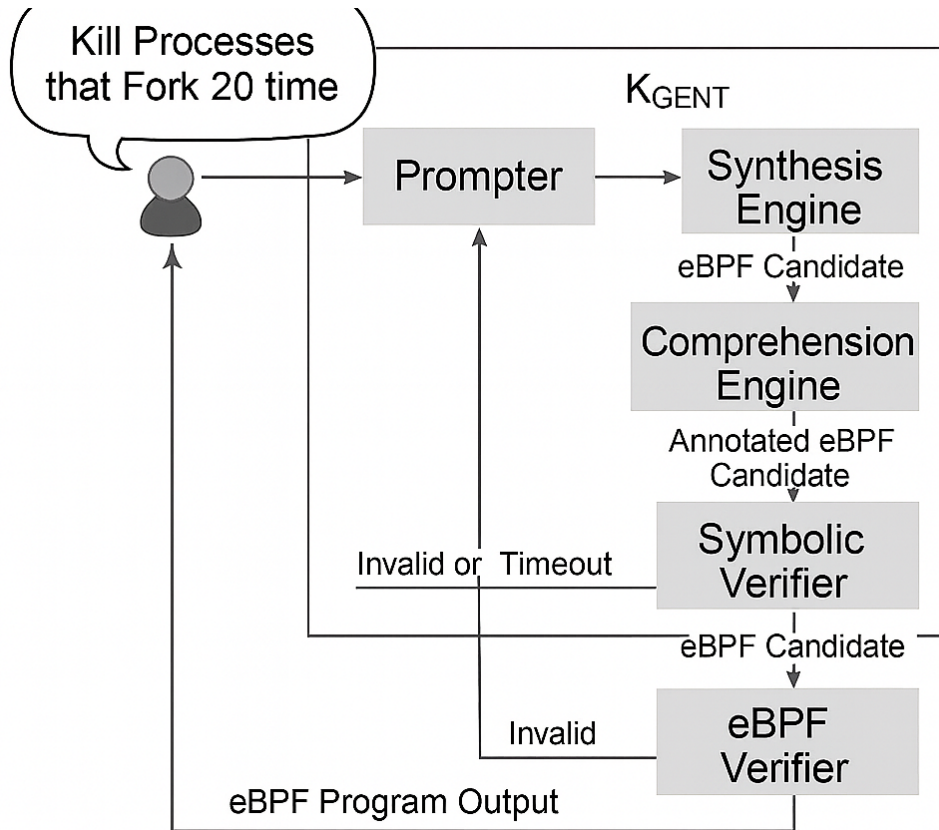


Figura 3.1: Workflow del sistema Kgent[13]

Ogni test case è stato processato attraverso il workflow completo di Kgent, comprendente le seguenti fasi:

1. **Analisi dell'intento:** parsing della richiesta utente ed estrazione di keywords per la query al vector database;
2. **Retrieval di esempi:** ricerca per similarità semantica nel database FAISS contenente esempi di programmi bpftime validati, con selezione dei 2–3 esempi più rilevanti;
3. **Costruzione del prompt:** assemblaggio di un prompt strutturato contenente:
 - istruzioni di sistema (“You are an expert in bpftime...”);

- esempi semplici predefiniti (few-shot learning);
 - esempi complessi recuperati dal vector database;
 - specifica del task dell'utente;
 - vincoli di output (“Write ONLY the bpfttrace program...”).
4. **Generazione del codice:** invio del prompt al modello LLM tramite API Ollama e ricezione del programma bpfttrace generato;
 5. **Post-processing:** estrazione del codice pulito dalla risposta del modello (rimozione markdown, commenti, testo esplicativo);
 6. **Validazione:**
 - compilazione con bpfttrace v0.16.0;
 - esecuzione con timeout di 20 secondi e privilegi `sudo`;
 - verifica del codice di ritorno.
 7. **Retry con feedback:** in caso di errore, il sistema incorpora il messaggio di errore completo nel prompt e richiede al modello di correggere il codice; sono consentiti fino a tre tentativi di correzione aggiuntivi dopo il primo fallimento.

Un test è considerato superato solo se il programma compila correttamente, si esegue per tutta la durata prevista senza andare in crash e non produce errori di runtime nello standard error.

3.1.4 Parametri di Generazione

Per garantire risultati comparabili e riproducibili, tutti i modelli sono stati eseguiti con configurazione identica:

```
model: ##
temperature: 0.1
max_tokens: 1024
stream: False
options:
  temperature: 0.1
  top_p: default
```

Listing 3.1: Parametri di generazione dei modelli

La temperatura bassa (0.1) garantisce output deterministici, privilegiando la correttezza rispetto alla creatività: in contesti critici come eBPF è essenziale evitare codice rischioso e favorire soluzioni stabili e sicure. Il limite di 1024 token è adeguato per i programmi bpfttrace che sono solitamente brevi.

3.2 Risultati

La Figura 3.2 riporta i risultati completi della valutazione sui 40 test case. DeepSeek-236B emerge come il modello più performante, raggiungendo un success rate del 92.5% (37 test superati su 40). I modelli di medie dimensioni (CodeLlama-13B, DeepSeek-16B) ottengono risultati comparabili, attestandosi intorno al 55-57.5% (22-23 test superati). CodeLlama-70B, nonostante le dimensioni superiori a 13B e 16B, ottiene un risultato simile (55.0%, 22 test), indicando che le dimensioni del modello non garantiscono automaticamente prestazioni migliori. Il modello più piccolo, CodeLlama-7B, raggiunge il 47.5% (19 test), un risultato comunque significativo considerando le sue dimensioni ridotte.

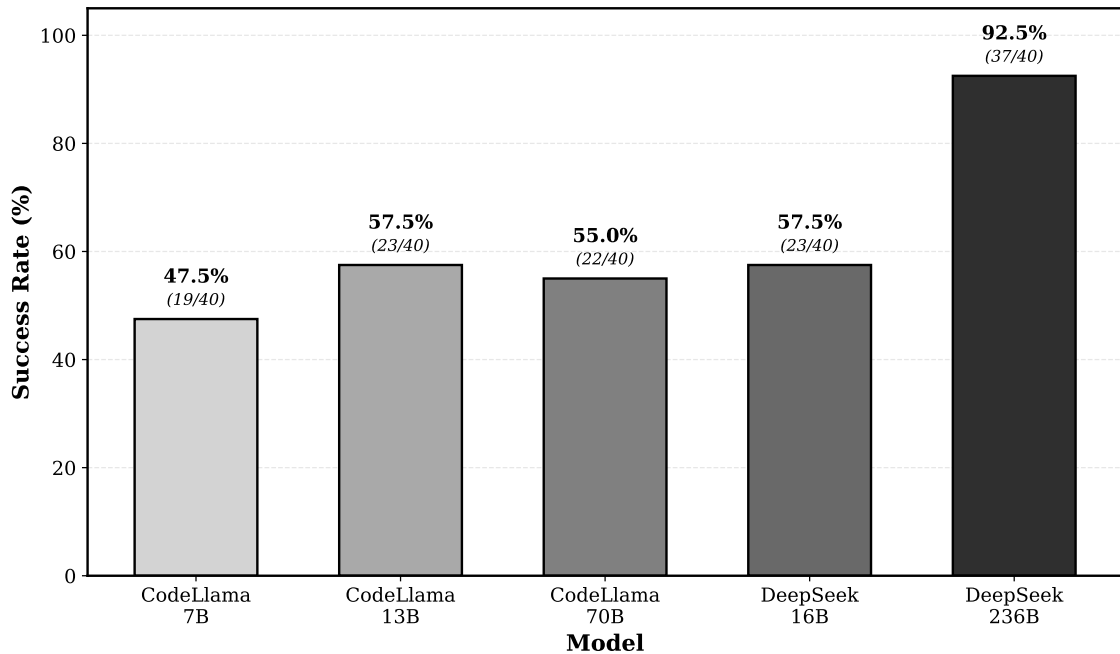


Figura 3.2: Tasso di successo per modello LLM

3.2.1 Tipologia di errori

L'analisi degli errori evidenzia pattern chiari:

- Nei modelli più piccoli prevalgono gli **errori sintattici**, dovuti a una conoscenza incompleta della sintassi di `bpfftrace`.
- Nei modelli di dimensioni maggiori, gli errori diventano prevalentemente **logici** o **semantici** (es. scelta errata di hook o condizioni di filtro).
- Non sono stati riscontrati **errori di ambiente** né di **verifica eBPF**, grazie al setup stabile della piattaforma di test.

Questo andamento mostra come l'aumento della complessità del modello porti a una comprensione più robusta della struttura del linguaggio, spostando l'errore da aspetti formali a concettuali.

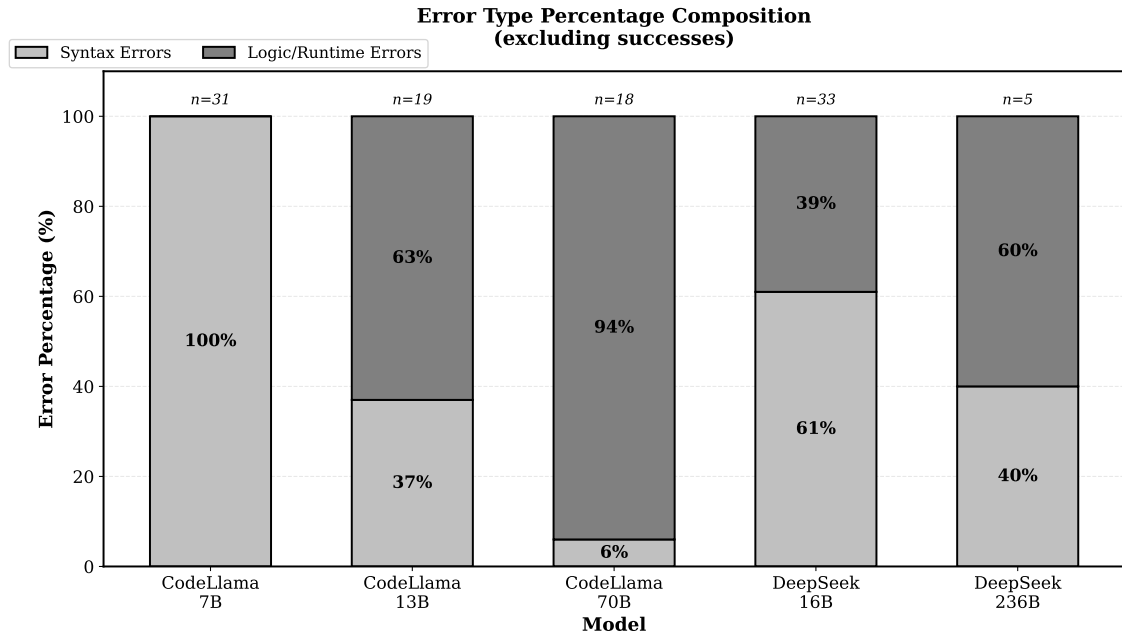


Figura 3.3: Distribuzione dei tipi di errore per modello

3.2.2 Effetto del retry

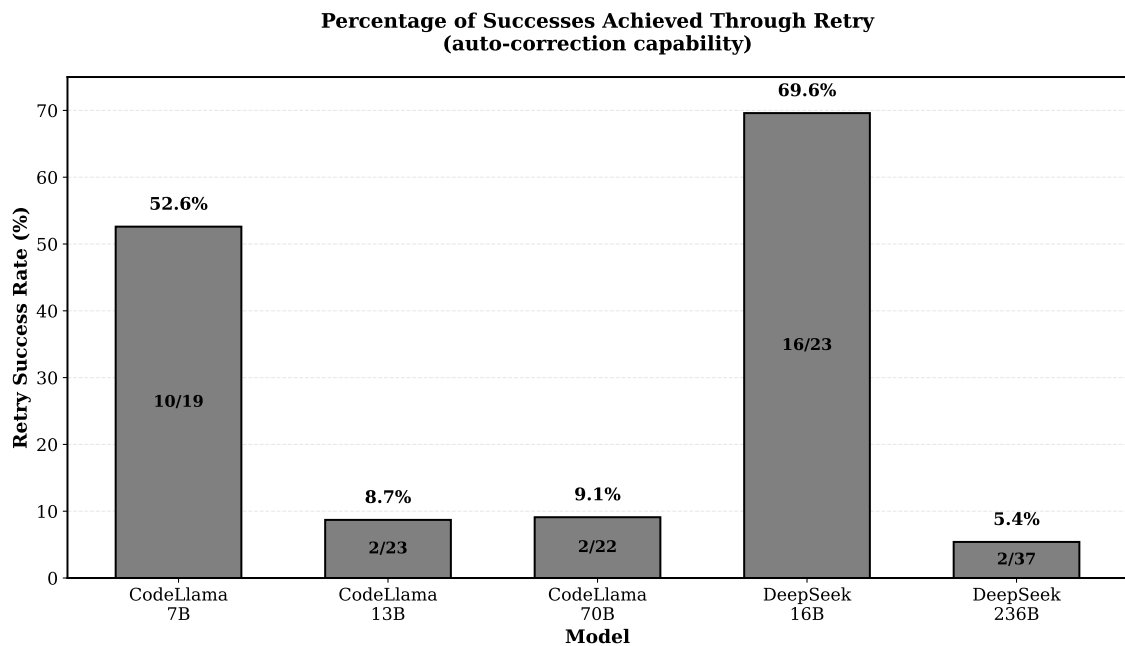


Figura 3.4: Percentuale di successi ottenuti tramite meccanismo di retry per ciascun modello.

Come si può notare dalla Figura 3.4, i modelli più piccoli beneficiano maggiormente dei retry. Il meccanismo di *retry* automatico, basato sul feedback dell'errore di compilazione, ha mostrato una buona efficacia. Nei modelli più grandi (CodeLlama-13B, CodeLlama-70B, DeepSeek-236B), circa il 5-9% dei successi totali è stato ottenuto dopo uno o più tentativi aggiuntivi, indicando una discreta capacità di auto-

correzione. Al contrario, i modelli più piccoli mostrano una dipendenza significativa dal retry: CodeLlama-7B ottiene il 52.6% dei propri successi tramite retry (10 test su 19), mentre DeepSeek-16B arriva al 69.6% (16 su 23), suggerendo che questi modelli tendono a fallire inizialmente ma possono correggersi efficacemente se guidati dal feedback di compilazione.

3.2.3 Performance per categoria di test

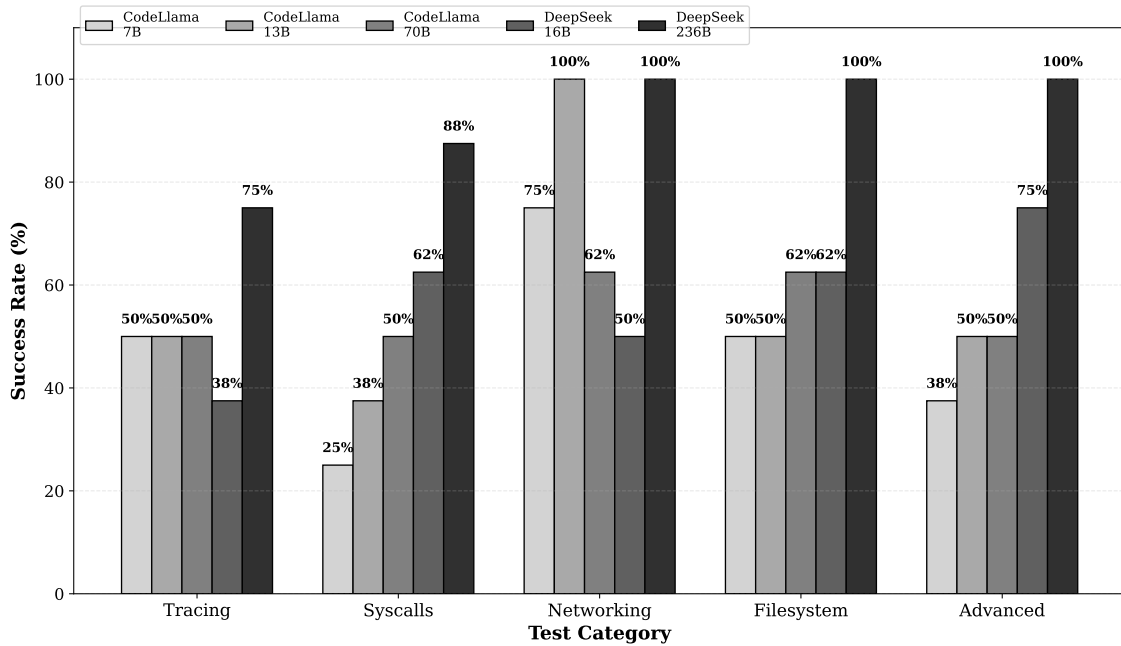


Figura 3.5: Success rate per categoria di test. Ogni categoria comprende 8 test.

Il dataset di test è suddiviso in cinque categorie principali, ciascuna composta da 8 test: *Tracing* (eventi kernel e sistema), *Syscalls* (chiamate di sistema), *Networking* (operazioni di rete), *Filesystem* (operazioni I/O su file) e *Advanced* (aggregazioni e analisi avanzate).

Come mostrato in Figura 3.5, DeepSeek-236B mantiene risultati elevati in tutti i gruppi, con tassi di successo compresi tra il 75% (*Tracing*) e il 100% (*Networking*, *Filesystem*, *Advanced*). Questo modello dimostra una capacità uniforme di gestire diverse tipologie di programmi eBPF, dalle operazioni di base alle aggregazioni complesse.

I modelli di medie dimensioni mostrano performance diversificate: CodeLlama-13B eccelle nel *Networking* (100%) ma fatica nei *Syscalls* (37.5%), mentre DeepSeek-16B ottiene risultati migliori negli *Advanced* (75%) e nei *Syscalls* (62.5%), segno di una maggiore capacità di gestire operazioni di sistema rispetto ai CodeLlama simili.

I modelli più piccoli mostrano limiti crescenti nei casi complessi: CodeLlama-7B raggiunge solo il 25% nei *Syscalls* e il 37.5% negli *Advanced*, ma ottiene il 75% nel *Networking*, risultato paragonabile a DeepSeek-236B, indicando che alcune categorie di test possono essere gestite efficacemente anche da modelli ridotti.

La variabilità tra categorie indica che la difficoltà dei task eBPF non dipende solo dalla loro complessità, ma anche dalla loro rappresentazione nei dataset di addestramento.

Capitolo 4

Conclusioni

Questo lavoro ha analizzato l'impiego di Large Language Models (LLM) per la generazione automatica di programmi eBPF tramite **bpfttrace**, realizzando un sistema agentico che integra retrieval semantico, few-shot learning e meccanismi di retry automatico.

La valutazione, condotta su un dataset di 40 test case rappresentativi, ha prodotto risultati significativi. Il modello **DeepSeek-236B** ha ottenuto il tasso di successo più elevato (92.5%, 37 test superati su 40), mostrando prestazioni consistenti in tutte le categorie. Un dato rilevante riguarda tuttavia le dimensioni del modello: **CodeLlama-70B** (55%, 22 test) non supera né **CodeLlama-13B** (57.5%, 23 test) né **DeepSeek-16B** (57.5%, 23 test). Ciò suggerisce che l'architettura e la qualità del processo di addestramento incidano in misura maggiore rispetto alla mera scala parametrica. Un incremento sostanziale delle prestazioni si osserva solo oltre la soglia dei 200 miliardi di parametri.

Il meccanismo di retry automatico risulta particolarmente efficace per i modelli di dimensioni ridotte, contribuendo al 52.6% dei successi in **CodeLlama-7B** (10 su 19) e al 69.6% in **DeepSeek-16B** (16 su 23). Nei modelli di grandi dimensioni l'impatto è invece marginale (5–9%), confermando che questi tendono a generare codice corretto al primo tentativo, mentre i modelli minori beneficiano maggiormente del feedback iterativo.

L'analisi per categoria evidenzia ulteriori pattern: **CodeLlama-7B** raggiunge il 75% nella categoria *Networking*, come **DeepSeek-236B**, ma si riduce al 25% nelle *Syscalls*. La difficoltà dei task sembra dipendere in misura maggiore dai dati di addestramento che dalla complessità intrinseca delle attività. Anche la tipologia di errore varia: i modelli più piccoli commettono prevalentemente errori sintattici (40% in CodeLlama-7B), mentre quelli di maggiore dimensione presentano errori logici (30% in CodeLlama-70B), come la selezione di hook point non corretti.

L'analisi comparativa con altri studi mostra che approcci basati su **GPT-4** ottengono risultati superiori, coerentemente con la scala del modello (circa 1.7 trilioni di parametri). Tuttavia, l'impiego di modelli proprietari presenta criticità rilevanti: costi elevati per chiamata API, latenza di rete, impossibilità di installazione on-premise e dipendenza da provider esterni. Per sistemi destinati a infrastrutture critiche, dove privacy e controllo dei dati risultano fondamentali, i modelli open-source rappresentano una scelta più appropriata, garantendo riproducibilità, autonomia e sostenibilità economica.

Dal punto di vista operativo, **DeepSeek-236B** risulta particolarmente adatto a contesti critici (tasso di fallimento inferiore al 10%), ma richiede circa 132 GB di RAM e quindi infrastrutture dedicate. I modelli di media scala, come **CodeLlama-13B** e **DeepSeek-16B**, costituiscono un compromesso efficace per ambienti edge o workstation convenzionali (55–57.5% di successo con 7–9 GB di memoria), mentre **CodeLlama-7B** (47.5%, 3.8 GB) risulta idoneo per prototipazione rapida in ambiti specifici.

Permangono tuttavia alcune limitazioni: il dataset è composto da soli 40 casi di valutazione, non è stata effettuata una validazione semantica automatica (un programma può compilare correttamente ma non eseguire il comportamento atteso), e nessun modello è stato sottoposto a fine-tuning specifico su eBPF, ovvero un processo di riaddestramento mirato che adatta un modello preesistente a un dominio o compito specifico mediante un insieme di dati specializzati. Migliorare il retrieval e integrare strategie multi-agente rappresentano direzioni di sviluppo promettenti.

In sintesi, la generazione automatica di programmi eBPF tramite LLM open-source risulta non solo fattibile ma anche efficace. **DeepSeek-236B** raggiunge livelli di affidabilità prossimi a contesti di produzione supervisionata, mentre modelli più compatti mantengono buone prestazioni in scenari mirati. L'evoluzione rapida dei modelli open-source lascia prevedere che, in un futuro prossimo, tali tecnologie diventeranno pienamente accessibili, abilitando l'utilizzo di eBPF anche da parte di sviluppatori non esperti di kernel programming.

Bibliografia

- [1] Eunomia bpf Community. Ken: Kernel event navigator (github repository). <https://github.com/eunomia-bpf/KEN>, 2024. Accessed: 2025-09-29.
- [2] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33:1877–1901, 2020.
- [3] Brendan Gregg. *BPF performance tools: Linux system and application observability*. Addison-Wesley Professional, 2019.
- [4] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997.
- [5] Steven McCanne and Van Jacobson. The BSD packet filter: A new architecture for user-level packet capture. In *USENIX Winter 1993 Conference Proceedings*, pages 259–270, 1993.
- [6] Tomas Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean. Efficient estimation of word representations in vector space. *arXiv preprint arXiv:1301.3781*, 2013.
- [7] Ollama Project. Ollama: Local large language model runner. <https://ollama.com/>, 2024.
- [8] Baptiste Roziere, Loubna Ben Allal, Leandro von Werra Li, Lewis Tunstall, Nathan Lambert, Alaaeldin Cassirer, Jonathan Tow, and et al. Code llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950*, 2023.
- [9] Baptiste Rozière, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Romain Sauvestre, Tal Remez, Jérémy Rapin, Artyom Kozhevnikov, Ivan Evtimov, Joanna Bitton, Manish Bhatt, Cristian Canton Ferrer, Aaron Grattafiori, Wenhan Xiong, Alexandre Défossez, Jade Copet, Faisal Azhar, Hugo Touvron, Louis Martin, Nicolas Usunier, Thomas Scialom, and Gabriel Synnaeve. Code llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950*, 2023.
- [10] Stuart J Russell and Peter Norvig. *Artificial Intelligence: A modern approach*. Pearson, 4th edition, 2020.
- [11] University of Turin and Polytechnic University of Turin. HPC4AI: High Performance Computing for Artificial Intelligence. <https://hpc4ai.unito.it/>.

Federated competence centre on High-Performance Computing, Artificial Intelligence and Big Data Analytics. Co-funded by Regione Piemonte via EU POR-FESR 2014-2020.

- [12] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in Neural Information Processing Systems*, 30, 2017.
- [13] Yiwei Yang, Yusheng Zheng, and Andrew Quinn. Kgent: Kernel extensions large language model agent. In *Proceedings of the ACM SIGCOMM 2024 Workshop on eBPF and Kernel Extensions*, eBPF '24, New York, NY, USA, August 2024. ACM.
- [14] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*, 2022.
- [15] Qihao Zhu, Daya Guo, Zhihong Shao, Dejian Yang, Peiyi Wang, Runxin Xu, Y. Wu, Yukun Li, Huazuo Gao, Shirong Ma, Wangding Zeng, Xiao Bi, Zihui Gu, Hanwei Xu, Damai Dai, Kai Dong, Liyue Zhang, Yishi Piao, Zhibin Gou, Zhenda Xie, Zhewen Hao, Bingxuan Wang, Junxiao Song, Deli Chen, Xin Xie, Kang Guan, Yuxiang You, Aixin Liu, Qiushi Du, Wenjun Gao, Xuan Lu, Qinyu Chen, Yaohui Wang, Chengqi Deng, Jiashi Li, Chenggang Zhao, Chong Ruan, Fuli Luo, and Wenfeng Liang. Deepseek-coder-v2: Breaking the barrier of closed-source models in code intelligence. *arXiv preprint arXiv:2406.11931*, 2024.

Ringraziamenti

Desidero dedicare queste righe a tutte le persone che, con impegno, disponibilità e professionalità, hanno reso possibile la realizzazione di questo lavoro.

Un ringraziamento al mio relatore, Professor Robert René Maria Birke, e al mio correlatore, Gianluca Mittone, per la disponibilità, il tempo e il supporto dimostrato durante lo stage, che mi ha permesso di affrontare temi avanzati e crescere professionalmente.

Ringrazio tutti i miei amici, di sempre e di oggi, per la compagnia, l'affetto e la presenza che non sono mai mancati. In questi tre anni di università mi hanno accompagnato in tantissimi momenti, tra viaggi, giornate spensierate, ore passate in biblioteca, al bar e durante le serate insieme. Ma soprattutto mi ricorderò sempre dei giri attorno a Saluzzo, che ho condiviso con ognuno di voi in momenti diversi, ma sempre speciali. Quei giri, pieni di chiacchiere, risate e pensieri, resteranno per me un ricordo indelebile. Un grazie speciale anche alle mie coinquiline, con cui ho condiviso la quotidianità e il mio primo anno a Torino. Sono esperienze che porterò sempre con me e che hanno reso questo percorso più leggero, autentico e pieno di bei ricordi.

Un ringraziamento particolare va ad Alessandro, Fabio, Damiano e a tutti i miei compagni di corso, con cui ho condiviso progetti, studio e tante giornate di lavoro: collaborare con loro è stato sempre stimolante e piacevole.

E, naturalmente, la mia famiglia — i miei genitori, Carla e Fabrizio — per la presenza costante, l'amore incondizionato e il sostegno che non è mai mancato. Un ringraziamento particolare a mio fratello, per l'aiuto, la vicinanza e la compagnia che mi ha offerto durante tutti questi anni di percorso, così come a tutte le persone che ho conosciuto lungo la strada e che hanno contribuito, anche solo con un sorriso, a rendere questo cammino speciale.

